Write-up

Fatura Masası · Sekiz labın çözümü

Her labda önce normal işlemi yapıyor, ardından bir HTTP isteğini değiştirerek iş kuralını sınıyoruz. Son adım, aynı denemeyi düzeltilmiş sürümde tekrarlamak ve normal işlevin çalışmaya devam ettiğini doğrulamak.

Anlatım boyunca üç soruyu ayır: **Hata nerede? Hangi koşulda kullanılabiliyor? Sonuçta ne oluyor?** SQL enjeksiyonu veya eksik nesne yetkilendirmesi hatanın türünü; izinsiz kayıt okuma, rol yükseltme veya kod çalıştırma teknik etkiyi anlatır. Fatura bilgilerinin sızması ve eksik tahsilat ise iş etkisidir.

Başlangıç

Repo kökünden uygulamayı çalıştır:

```
python -m pip install -r mini-lab/requirements.txt
python mini-lab/app.py
```

Tarayıcıda `http://127.0.0.1:5050` adresini aç. Ders hesabı olarak **Alice**, sürüm olarak **Zafiyetli sürüm** seçili olsun. Temiz bir gösterim için başlangıçta **Verileri sıfırla** düğmesini kullan. Bu işlem iki sürümün verilerini ve bütün lab ilerlemesini temizler.

İstek örnekleri uygulamadaki **İstek masası** üzerinden gönderilir. Tarayıcı, çalışma alanını belirleyen oturum çerezini kendisi taşır. Harici bir HTTP istemcisi kullanırsan aynı çerezi de taşımalısın; başka tarayıcı oturumu ayrı veri ve JWT anahtarı kullanır.

Zafiyetli kod paneli, çalışan `app.py` dosyasının zafiyetli dalından türetilen kısa örneği gösterir. Sürüm seçme koşulları ve lab ilerleme kayıtları örnekten çıkarılır. Düzeltilmiş sürüme geçildiğinde de bu kod kalır; karşılaştırma istek ve yanıt üzerinden yapılır.

Hesaplar ve kayıtlar

Hesap	Şirket ve rol	Normal erişim
Alice	A şirketi, üye	Kendi 101 ve 104 numaralı faturaları
Bob	A şirketi, üye	Kendi 102 numaralı faturası
Deniz	A şirketi, muhasebeci	A şirketinin 101, 102 ve 104 numaralı faturaları
Ece	B şirketi, üye	Kendi 103 numaralı faturası

Hesap seçicisi farklı izinleri göstermek içindir; gerçek bir giriş mekanizması değildir. Kupon sepeti, ürün siparişi ve 501 numaralı ödeme örneği birbirinden bağımsızdır. İlerleme işaretleri gözlenen etkiyi takip eder; sınav notu olarak kullanılmaz.

Hızlı başvuru

Lab	Deneme	Düzeltilmiş davranış
1 · Arama	Aramayla kapsam dışı faturaları getir	Aynı metin sorguyu değiştirmez; sonuç boş
2 · Fatura erişimi	Alice ile 102 numaralı kaydı aç	`403 Forbidden`
3 · Alan izinleri	Not isteğine `status` ekle	`403`; hiçbir alan değişmez
4 · Teslimat notu	Notta JavaScript çalıştır	Girdi metin olarak görünür
5 · Ürün fiyatı	900 TL'lik ürüne 1 TL gönder	`201`; tutar sunucuda 900 TL hesaplanır
6 · Kupon	Aynı indirimi ikinci kez uygula	`409`; toplam 800 TL kalır
7 · Ödeme akışı	Ödemeden teslimat onayı iste	`409 payment_required`
8 · JWT	Token'ın kullanıcı/rol alanını değiştir	Aynı token için `401 invalid_token`

1. Arama üzerinden SQL enjeksiyonu

Normal akış ve deneme

Arama çalışmasını aç. `ofis` araması Alice'in 104 numaralı faturasını döndürür. Boş aramada yalnız 101 ve 104 görünür. Beklenen kural, arama sonuçlarının kullanıcının izinleriyle sınırlı kalmasıdır.

Arama kutusuna şu değeri yaz:

```
%' OR 1=1 --
```

Ara düğmesine bas. Yanıtta `count: 4` ve 101, 102, 103, 104 numaralı kayıtlar görünür. Bob'un 102 numaralı faturası ve B şirketinin 103 numaralı faturası artık Alice'e dönmektedir. İstek masasında aynı denemenin URL kodlanmış biçimi şöyledir:

```
GET /api/vulnerable/invoices/search?q=%25%27%20OR%201%3D1%20--%20
```

Kök neden

Zafiyet türü **SQL injection / SQL enjeksiyonu**. Arama girdisi sorguya parametre olarak verilmek yerine SQL metnine ekleniyor:

```
sql = f"SELECT * FROM invoices WHERE {scope} AND title LIKE '%{query}%' ORDER BY id"
```

Girdideki tek tırnak metin değerini kapatır. `OR 1=1` koşulu sorguya yeni bir mantıksal dal ekler; `--` kalan kısmı yorum yapar. Koşulun ilgili bölümü şuna dönüşür:

```
WHERE company = ? AND owner_id = ? AND title LIKE '%%'
OR 1=1 -- %' ORDER BY id
```

`AND`, `OR` işleminden önce değerlendirilir. Böylece önceki izin koşullarını sağlamayan satırlar da `OR 1=1` nedeniyle seçilebilir. Şirket ve kullanıcı değerlerinin parametre olması, sorgunun başka bir parçasındaki metin birleştirme hatasını ortadan kaldırmaz.

Düzeltmeyi doğrulama

Düzeltilmiş sürüm seç; düzenlenen istek korunur. Aynı isteği gönder. Yanıt `200` olur, fakat `count: 0` döner. Bu beklenen sonuçtur: saldırı metni artık aranacak açıklama değeridir.

```
sql = f"SELECT * FROM invoices WHERE {scope} AND title LIKE ? ORDER BY id"
params.append("%" + query + "%")
```

Son olarak `ofis` aramasını yeniden dene; 104 numaralı kayıt bulunmalıdır.

Tür → etki: Metin birleştirmeyle SQL enjeksiyonu → izin kapsamını aşan kayıt okuma → finansal bilgilerin ifşası. Bu labda sorgu, her istekte oluşturulan ayrı bir bellek veritabanında çalışır; ana uygulama deposuna erişim veya sunucuda kod çalıştırma gösterilmez.

Derste sor: “Sonuç sayısının artması tek başına kanıt mı?” Beklenen cevap, sonuçlarda Alice'in erişememesi gereken kayıtların gösterilmesidir.

2. Fatura kimliğini değiştirerek izinsiz erişim

Normal akış ve deneme

Fatura erişimi çalışmasında Alice'in 101 numaralı faturasını aç. Ardından numarayı `102` yap:

```
GET /api/vulnerable/invoices/102
```

Yanıt `200` olur. Yanıtın ilgili alanları:

```
{
  "invoice": {
    "id": 102,
    "owner_id": "bob",
    "company": "A"
  }
}
```

İstek başarılı olmasına rağmen ders hesabı hâlâ Alice'tir. Kayıt Bob'a aittir. Aynı şirkette bulunmak, üyeler arasında otomatik okuma yetkisi oluşturmaz.

Kök neden

Burada **IDOR / Insecure Direct Object Reference (Güvensiz Doğrudan Nesne Referansı)**, kimliğin değiştirilerek izinsiz nesneye ulaşılmasını tarif eder. API bakımından eksik kontrol **BOLA / Broken Object Level Authorization (Nesne Düzeyinde Yetkilendirme Eksikliği)** olarak ele alınır.

Sunucu faturayı kimlikle buluyor, fakat zafiyetli GET yolunda `can_read` sonucunu erişimi engellemek için kullanmıyor. Listede yalnız Alice'in faturalarını göstermek, doğrudan ayrıntı adresini korumaz.

Düzeltmeyi doğrulama

Aynı 102 isteğini düzeltilmiş sürüme gönder:

```
GET /api/fixed/invoices/102
```

Beklenen yanıt `403` ve `error: forbidden`. Kendi 101 numaralı faturası hâlâ `200` dönmelidir. Ek karşılaştırma için Deniz hesabını seç: muhasebeci 102'yi okuyabilir, B şirketine ait 103'ü okuyamaz.

```
return (
    invoice["company"] == user["company"]
    and (invoice["owner_id"] == user["id"]
         or user["role"] == "accountant")
)
```

Bu kontrol her ayrıntı isteğinde uygulanır. Kimlik sayı yerine UUID olsa da kullanıcının bu kaydı okuyup okuyamayacağı yine denetlenmelidir.

Tür → etki: Eksik nesne yetkilendirmesi → başka kullanıcının faturasını okuma → finansal gizliliğin ihlali. Bu örnekte veri değiştirme gösterilmez; PATCH yolunun ayrıca izin kontrolü vardır.

Derste sor: "102 yerine tahmin edilemeyen bir kimlik kullansaydık sorun çözülür müydü?" Beklenen cevap, tahmin zorluğunun izin kontrolünün yerine geçmediğidir.

3. Not güncellemesiyle fatura onaylama

Normal akış ve deneme

Alice'in 101 numaralı faturası başlangıçta `draft` durumundadır. **Alan izinleri** ekranından yalnız teslimat notunu güncelle; normal PATCH isteğini incele. Sonra aynı gövdeye `status` ekle:

```
PATCH /api/vulnerable/invoices/101
Content-Type: application/json

{"note":"Yarın teslim","status":"approved"}
```

Yanıt `200` olur ve `invoice.status` artık `approved` görünür. Alice yalnız üye olmasına rağmen fatura onaylanmıştır.

Kök neden

Zafiyet **BOPLA / Broken Object Property Level Authorization (Nesne Alanı Düzeyinde Yetkilendirme Eksikliği)** kapsamındadır. Toplu alan atamada izin verilen alanlar listesi fazla geniştir:

```
allowed = {"note", "status"} if mode == "vulnerable" else {"note"}
invoice.update({k: v for k, v in payload.items() if k in allowed})
```

Alice'in nesneye erişimi vardır; fatura gerçekten kendisine aittir. Hata, aynı erişimin bütün alanları değiştirme yetkisi gibi değerlendirilmesidir. `status` güncellemesi, not düzenleme işleminden farklı bir iş yetkisi gerektirir.

Düzeltmeyi doğrulama

İki alanı içeren gövdeyi düzeltilmiş sürüme gönder. Yanıt `403 field_forbidden` olmalı. Ardından 101'i tekrar oku: durum `draft`, not ise başlangıç değerinde kalmalıdır. Reddedilen istek kısmen uygulanmaz.

Yalnız şu gövdeyi gönderdiğinde `200` alınmalıdır:

```
{"note":"Yarın teslim"}
```

Düzeltme, bu işlemde yalnız `note` alanına izin verir. Gerçek bir onay işlemi ayrıca rolü, şirket kapsamını, tutar limitini ve mevcut durumu doğrulamalıdır; bu uygulamada ayrı bir fatura onay servisi gösterilmez.

Tür → etki: Fazla geniş alan izni → onay durumunun yetkisiz değiştirilmesi → muhasebe onay sürecinin aşılması.

Derste sor: “Fatura kendisinin olduğuna göre hangi izin eksik?” Beklenen cevap, nesne izni ile alan/işlem izninin ayrı olduğudur.

4. Teslimat notunda kalıcı XSS

Normal akış ve deneme

Teslimat notu ekranında sıradan bir metin kaydet. Ardından aşağıdaki girdiyi not alanına yazıp yeniden kaydet:

```
<img src=x onerror="alert('Not içindeki kod çalıştı')">
```

Sunucuya giden isteğin biçimi:

```
POST /api/vulnerable/notes
Content-Type: application/json

{"note":"<img src=x onerror=\"alert('Not içindeki kod çalıştı')\">"}
```

Not önizlemesinde “Çalışan kodun mesajı: Not içindeki kod çalıştı” görünür. Labın `alert` işlevi bu mesajı önizlemeye yazar; ayrı bir tarayıcı iletişim kutusu açmaz. Aynı çalışmayı yeniden açmak, saklanan notun tekrar işlendiğini gösterir.

Kök neden

Zafiyet **stored XSS / stored cross-site scripting (Kalıcı Siteler Arası Betik Çalıştırma)**. Kaydedilen kullanıcı verisi HTML çıktısına kaçış uygulanmadan eklenir:

```
rendered = note if mode == "vulnerable" else str(escape(note))
```

Tarayıcı `<img>` öğesini HTML olarak işler. Görsel yüklenemeyince `onerror` çalışır. Önizlemede ağ erişiminin CSP ile engellenmesi, hata olayının oluşmasını engellemez; olay işleyicisi JavaScript çalıştırır.

Düzeltmeyi doğrulama

Düzeltilmiş sürümde aynı notu ayrıca kaydet; sürümlerin notları ayrıdır. Bu kez girdi metin olarak görünmeli, “Çalışan kodun mesajı” oluşmamalıdır. Normal bir metin notu kaydedip tekrar açarak işlevin korunduğunu da göster.

Bu çıktı bağlamı için düzeltme HTML kaçışıdır. Ürün gerçekten zengin metin kabul edecekse, izin verilen HTML öğelerini uygulayan güvenilir bir temizleme politikası ayrıca tasarlanır.

Tür → etki: Güvensiz HTML çıktısı → tarayıcıda JavaScript çalışması → görüntülenen içeriğin veya kullanıcı işlemlerinin etkilenmesi. Buradaki önizleme sandbox içinde çalışır ve ağ erişimi kapalıdır. Gösterilen kanıt notun kaydedilip önizlemede çalışmasıdır; başka kullanıcının oturumunun ele geçirildiği veya sunucuda RCE elde edildiği sonucuna varılmaz.

Derste sor: “Notu veritabanına yazmış olmak XSS kanıtı mı?” Beklenen cevap, kaydetme ile tarayıcıda kod çalışmasının farklı aşamalar olduğudur.

5. İstemci fiyatıyla ucuz sipariş oluşturma

Normal akış ve deneme

Ürün fiyatı ekranında USB-C çoklayıcı için bir sipariş oluştur. Katalog fiyatı 900 TL; normal istekte `unit_price_cents` değeri `90000` görünür. Tutarlar API'de tam sayı kuruştur.

Birim fiyatı `100` yaparak yeniden gönder:

```
POST /api/vulnerable/orders
Content-Type: application/json

{"product_id":"hub","quantity":1,"unit_price_cents":100}
```

Yanıt `201 Created` olur. Siparişte `unit_price_cents: 100` ve `total_cents: 100` döner: 900 TL'lik ürün 1 TL'ye sipariş edilmiştir.

Kök neden

Bu bir **business logic flaw / iş mantığı hatası**. Sunucu, maliyet hesabının yetkili kaynağı olarak istemciden gelen fiyatı kullanır:

```
unit = payload.get("unit_price_cents", product["price_cents"])
```

Bir alanın formda düzenlenememesi veya JavaScript ile sabitlenmesi HTTP isteğini korumaz. İstek masası, tarayıcının ağ araçları veya bir proxy ile alan değiştirilebilir. Bu hatayı göstermek için SQL ya da JavaScript kodu enjekte etmek gerekmez.

Düzeltmeyi doğrulama

Aynı 100 kuruşluk gövdeyi düzeltilmiş sürüme gönder. Yanıt yine `201` olmalıdır; bu kez `unit_price_cents: 90000` ve `total_cents: 90000` döner. Başarı ölçütü isteğin reddedilmesi değil, fiyatın doğru kaynaktan hesaplanmasıdır.

```
unit = product["price_cents"]
```

Adedi `2` yaparsan düzeltilmiş sürüm toplamı `180000` kuruş hesaplar. Adet her iki sürümde de 1–10 arası tam sayı olmalıdır; negatif adet bu labın zafiyeti değildir.

Tür → etki: İstemci fiyatına güvenme → sipariş tutarının manipülasyonu → gerçek bir ticari akışta eksik tahsilat. Labda gösterilen sonuç sipariş kaydının tutarıdır; gerçek bir ödeme hareketi yapılmaz.

Derste sor: “İki sürüm de 201 döndürüyorsa düzeltmeyi nasıl anlayacağız?” Beklenen cevap, durum koduna ek olarak sunucunun hesapladığı tutarı karşılaştırmaktır.

6. Tek kullanımlık kuponun tekrarlanması

Normal akış ve deneme

İndirim kuponu ekranındaki bağımsız sepet 1000 TL ile başlar. `MERHABA20`, kullanıcı başına bir kez %20 indirim sağlamalıdır. Kuponu normal düğmeyle uygula: tutar 800 TL olur ve düğme kapanır.

İstek masasında aynı POST isteğini tekrar gönder:

```
POST /api/vulnerable/coupons/apply
Content-Type: application/json

{"code":"MERHABA20"}
```

İkinci yanıt da `200` olur. `total_cents` değeri `64000`, yani sepet tutarı 640 TL'dir. Yeni indirim ilk tutara değil, mevcut 800 TL'ye uygulanmıştır.

Kök neden

Hata, **tek kullanım kuralının sunucuda uygulanmaması**. Zafiyetli sürüm kullanım bilgisini kaydeder, fakat indirimden önce bu bilgiyi engelleyici koşul olarak kullanmaz. Arayüzde düğmenin kapanması yalnız kullanıcı deneyimidir.

```
used = uid in coupon["used_by"]
if mode == "fixed" and used:
    raise LabError("Bu kuponu daha önce kullandınız.", 409, "already_used")
coupon["totals"][uid] = coupon["totals"][uid] * 80 // 100
```

Bu gösterim aynı isteğin sırayla tekrarlanmasıdır. Eş zamanlı iki isteğe ihtiyaç yoktur; bulguyu yarış koşulu diye adlandırmak gerekmez.

Düzeltmeyi doğrulama

Düzeltilmiş sürümün temiz sepetine ilk uygulama `200`, ikinci uygulama `409 already_used` dönmelidir. Sepet 800 TL'de kalır. Bob hesabına geçip ilk kullanımı denersen kabul edilmelidir; kural bütün kullanıcılar için tek kullanım değil, kullanıcı başına tek kullanımdır.

Sunucuda kullanım kontrolü, tutar güncellemesi ve kullanım kaydı aynı veritabanı işlemi içinde yapılır. Gerçek bir sistemde kullanıcı/kampanya anahtarının benzersizliği ve eş zamanlı işlemlerin davranışı da korunmalıdır.

Tür → etki: Kampanya kuralının eksik uygulanması → aynı indirimin tekrar alınması → gelir kaybı ve kampanya maliyetinin artması.

Derste sor: “Buton kapalıysa istek nasıl gitti?” Beklenen cevap, arayüz kısıtının doğrudan API çağrısını engellemediğidir.

7. Ödeme adımını atlayarak teslimat onayı

Normal akış ve deneme

Ödeme akışı ekranında Alice'in 501 numaralı siparişi başlangıçta `payment_status: unpaid`, `status: awaiting_payment` durumundadır. Normal sıra şöyledir:

```
Sipariş → Ödeme doğrulandı → Teslimat onayı
```

Ödeme düğmesine basmadan, istek masasındaki onay isteğini gönder:

```
POST /api/vulnerable/orders/501/confirm
Content-Type: application/json

{}
```

Yanıt `200` olur. Aynı kayıtta iki alanı birlikte göster:

```
{"payment_status":"unpaid","status":"confirmed"}
```

Bu çelişki kanıttır: siparişin teslimat onayı vardır, ödeme kaydı yoktur.

Kök neden

Zafiyet **workflow bypass / iş akışı adımını atlama**. Kullanıcı siparişin sahibi olduğu için nesne erişimi uygundur. Eksik olan, onay durumuna geçişin ön koşuludur. Arayüzün onay düğmesini kapatması sunucunun durum makinesini korumaz.

Düzeltmeyi doğrulama

Aynı isteği düzeltilmiş sürüme gönder: `409 payment_required` dönmelidir. Sonra normal sırayı tamamla:

```
POST /api/fixed/orders/501/pay
Content-Type: application/json

{}
```

Ödeme kaydından sonra `/api/fixed/orders/501/confirm` çağrısı `200` dönmelidir. Kayıt artık `paid` ve `confirmed` değerlerini birlikte taşır. Tekrar onay isteği `409` ile reddedilir. Bob'un bu siparişi onaylaması da sahiplik kontrolü nedeniyle `403` alır.

Labın **Deneme ödemesi yap** işlemi gerçek bir ödeme sağlayıcısını temsil eden bir simülasyondur. Üretimde ödeme doğrulaması, istemcinin "ödedim" beyanından değil, güvenilir sunucu tarafı ödeme kaydından gelmelidir.

Tür → etki: Eksik durum geçişi kontrolü → ödemesiz teslimat onayı → gerçek bir süreçte tahsilatsız mal/hizmet teslimi.

Derste sor: "Bu kayıt Alice'in olduğu hâlde neden işlem reddedilmeli?" Beklenen cevap, sahipliğin iş akışındaki ön koşulları kaldırmadığıdır.

8. JWT payload değiştirerek kimliğe bürünme

Normal token ve değiştirme

Alice hesabıyla **JWT ile kimlik** çalışmasını aç. **Yeni ders token'ı al**, ardından **Faturaları bu token'la getir** düğmesine bas. `200` yanıtında 101 ve 104 numaralı faturalar görünür.

JWT, `header.payload.signature` biçiminde üç parçadır. Header'da `alg: HS256` görülür. Payload'daki `sub` kullanıcıyı, `role` rolü, `company` şirketi belirtir. `iss`, `aud`, `iat`, `exp` alanları sırasıyla üretici, hedef, üretim zamanı ve bitiş zamanıyla ilgilidir.

Payload editöründe yalnız `sub` değerini `alice` yerine `bob` yap; diğer alanları koru. **Payload'u token'a yaz** düğmesine bas. İlk parça ve imza korunurken ortadaki parça değişir. Yeni bir imza üretilmemiştir.

```
GET /api/vulnerable/jwt/invoices
Authorization: Bearer <DEGISTIRILMIS_DERS_TOKENI>
```

Faturaları bu token'la getir düğmesine basınca `200` alınır. Sunucunun kabul ettiği kimlik Bob, dönen fatura 102'dir. Üstteki ders hesabının Alice olarak kaldığını göster: değişen bilgi hesap seçicisi değil, token'daki iddiadır.

Kök neden

Zafiyet, **JWT imza doğrulamasının yapılmaması**. Zafiyetli dal token'ı yalnız çözer:

```
claims = jwt.decode(token, options={"verify_signature": False})
```

Base64url ile kodlanmış JSON'u okumak, bu verinin güvenilir üreticiden geldiğini kanıtlamaz. Sunucu doğrulanmamış `sub`, `role` ve `company` değerlerini erişim kararında kullanınca saldırgan kendi kimliğini veya yetkisini belirleyebilir.

Aynı token ile düzeltme kontrolü

Değiştirilmiş token'ı koruyarak **Düzeltilmiş sürüm** seç ve tekrar gönder. Yanıt `401 invalid_token` olmalıdır. Bu aşamada yeni token almak, karşılaştırılan girdiyi değiştirir; önce aynı token'ın reddini göster.

```
claims = jwt.decode(
    token, key, algorithms=["HS256"],
    issuer="fatura-masasi", audience="fatura-api",
    options={"require": ["sub", "role", "company", "iss", "aud", "exp", "iat"]},
)
```

Ardından yeni bir orijinal Alice token'ı alıp düzeltilmiş sürümde gönder: normal işlem `200` dönmelidir. Token'lar 15 dakika geçerlidir. Uzun süren bir gösterimde başlangıç token'ını yenilemek, süre dolmasını imza değişikliğiyle karıştırmayı önler.

İkinci deneme olarak orijinal Alice token'ında yalnız `role` değerini `accountant` yap. Zafiyetli sürüm 101, 102 ve 104'ü getirir; düzeltilmiş sürüm yine `401` döndürür. Bu, kullanıcı değiştirmeden rol yükseltmenin aynı eksik doğrulamadan doğduğunu gösterir.

jwt_tool ile aynı deneme

Aracı kurduysan uygulamadan ders token'ını kopyalayıp terminalde çözebilir ve değiştirebilirsin:

```
$labJwt = 'UYGULAMADAN_KOPYALANAN_DERS_TOKENI'
python jwt_tool.py "$labJwt"
python jwt_tool.py "$labJwt" -T
```

Çıktıdaki değiştirilmiş token'ı web ekranındaki **Token** alanına yapıştır. **Token'ı çöz**, ardından **Faturaları bu token'la getir** düğmesini kullan. Terminaldeki çözümleme tek başına bulgu değildir; kanıt, sunucunun değiştirilmiş token ile izinsiz faturayı döndürmesidir. Araç kurulumu için jwt_tool deposu ve uygulama notu kullanılabilir.

Tür → etki: Eksik kriptografik doğrulama → kimliğe bürünme veya rol yükseltme → izinsiz finansal veri erişimi. Anahtar tarayıcıya verilmez; bu lab anahtar kırmayı göstermez. Anahtar çalışma alanına özeldir; aynı karşılaştırmayı aynı tarayıcı oturumunda yap. Sıfırlama anahtarı değiştirir ve eski token düzeltilmiş sürümde geçersiz olur.

Derste sor: "Payload'u geri değiştirirsek token yeniden geçerli olur mu?" Orijinal imzalanan baytlar birebir geri gelirse olabilir; JSON alan sırası ve kodlama da imzanın girdisidir. Temiz karşılaştırma için yeni orijinal token almak daha anlaşılırdır.

Kapanış notları

Her labın sonunda öğrenciden kısa bir bulgu anlatımı iste: **değiştirdiği istek, gördüğü kanıt, eksik kontrol ve aynı denemenin düzeltme sonrası sonucu**. Durum kodunu tek başına başarı ölçütü sayma. Arama ve fiyat örneklerinde düzeltilmiş sürüm de başarılı HTTP yanıtı verir; önemli olan kapsam ve tutardır.

Karıştırılabilecek üç ayrımı özellikle sor:

- Nesneye erişebilmek, nesnenin bütün alanlarını değiştirebilmek değildir.
- Kimlik ve rol doğrulaması, fiyat/kupon/ödeme gibi iş kurallarını kendiliğinden sağlamaz.
- Kod çalışmasının nerede olduğu önemlidir: XSS tarayıcı bağlamını, RCE hedef uygulamanın çalıştığı sistemdeki kod yürütme etkisini anlatır. Bu labların SQLi veya JWT sonucundan doğrudan RCE sonucu çıkarılmaz.

İş etkisini kanıttan ayrı kur: labda 1 TL'lik sipariş kaydı oluşması gözlenen teknik sonuçtur; gerçek bir ödeme ve teslimat sisteminde bunun eksik tahsilata dönüşmesi, ilgili entegrasyonların da aynı tutara güvenmesine bağlıdır.

Kaynak uygulama app.py, görüntülenen kod parçaları code_views.py, davranış kontrolleri test_app.py dosyalarındadır.